Lehrerexemplar · Boss 6 – Generalprobe · Welt 6 · DS 37

Erwartungshorizont: Probeklausur „Die Paketstation“

Erwartungshorizont / Bewertungsbogen für _____

I, II, III: erreichbare Bewertungseinheiten im jeweiligen Anforderungsbereich · BE: vom Prüfling erreichte Bewertungseinheiten

Erwartete Prüfungsleistungen	I	II	III	BE
a) Konstruktoren und Kapselung erläutern Kernpunkte: Konstruktor erzeugt und initialisiert ein Objekt; Paketstation(standort): Station ohne Fächer mit leerer Reihung, Schlange und leerem Stapel; Paketstation(standort, anzahlS, anzahlM, anzahlL): zusätzlich Fächer je Größe (Abb. 2: 101 – 107), Überladung; „–“ = privat, Zugriff über die öffentliche Operation getInhalt() und dann getNummer().	3	1		
b) Implementierung: Suche in einer dynamischen Reihung von Objekten Kernpunkte: Methodenkopf int sucheFach(String sendungsnr); Durchlauf mit Index 0 bis getLength() − 1; Cast (Fach); istFrei() vor getInhalt(); Vergleich mit equals; Rückgabe der Fachnummer (nicht des Index) beim ersten Treffer; −1 erst nach vollständigem Durchlauf, Rückgabe in jedem Pfad, leere Station ohne Laufzeitfehler.	1	4	2	
c) Struktogramm: Schlange auswerten und unverändert lassen Kernpunkte: Kopf mit Signatur; Zähler ← 0; Hilfsschlange; solange nicht auftraege.isEmpty(): dequeue, Größe vergleichen, zählen, in Hilfsschlange einreihen; danach alle zurückschichten; gib anzahl zurück; Notation nach EH §5.	1	4	1	
d) Tracetabelle mit Schlange, dynamischer Reihung und Stapel Kernpunkte: Durchläufe: P-4127 → Fach 104; P-9054 → Fach 107; P-1863 → Fach 102; P-6620 → Fach 105; P-3089 → Rücksendung; P-8845 → Rücksendung; ruecksendungen am Ende (oben zuerst) P-8845 (L), P-3089 (M); Rückgabe 4.	2	2		
e) Veränderung eines Algorithmus erläutern Kernpunkte: Erst wie bisher nach freiem Fach gleicher Größe suchen; ohne Treffer erneut mit der nächstgrößeren Größe (S → M → L) suchen, erst danach Rücksendung; kleinstmögliches Fach durch größenweise aufsteigende Suche (oder Minimum über alle passenden freien Fächer).		1	1	
f) Rekursionstabelle und Prinzip der Rekursion Kernpunkte: Tiefen 0 bis 4 mit platz 8, 6, 5, 2, 1; Abbruch, weil P-1190 (M) nicht mehr passt (2 > 1); Rückgaben 0, 1, 2, 3, 4 (von innen nach außen), Ergebnis 4; Prinzip: Selbstaufruf mit kleinerem Problem, zwei Basisfälle, Ergebnis 1 + Ergebnis des Aufrufs.	2	3		
g) Rekursionstiefe begründen Kernpunkte: Selbstaufruf nur, wenn das oberste Paket passt (Volumen ≤ platz); jedes Volumen ≥ 1, also sinkt platz je Ebene um mindestens 1 und wird nie negativ → Tiefe t erfüllt t ≤ platz; Gleichheit nur bei lauter S-Paketen.		1	1	
h) Fehleranalyse einer rekursiven Methode Kernpunkte: Fehler 1 (Zeile 10): Ergebnis des rekursiven Aufrufs nicht verwendet → nur Fach i zählt (Fach 101 belegt → 0); Fehler 2 (Zeile 2): Basisfall i == getLength() − 1 → letztes Fach 107 nie geprüft; Korrektur: Basisfall i == getLength() (0), Ergebnis addieren; leere Station: getItem(0) → Laufzeitfehler (IndexOutOfBoundsException), korrigiert 0.	1	3	1	
Summe	10	19	6	**/ 35**

AFB-Verteilung: I 10 BE (29 %) · II 19 BE (54 %) · III 6 BE (17 %) – Abitur-Richtwert 30 / 53 / 17 %. Zeitbudget: 35 BE · 2,5 min ≈ 88 min (+ Reserve) = 90 min.

Die gewählten Lösungsansätze und -wege müssen nicht mit den dargestellten Lösungen identisch sein. Fachlich richtige Alternativen sind unter Berücksichtigung der verbindlichen Vorgaben (EH 2025) entsprechend zu bewerten. Laufzeitfehler, die nicht durch explizite Abfragen ausgeschlossen werden, führen zu Abzug (EH §4). Die Kürzel P1 – P10 verweisen auf die „Top 10 Punktverluste“ (AB1 Korrektur und Fehlerjagd).

Orientierung Notenpunkte (informativ)

Notenpunkte	15	14	13	12	11	10	9	8
ab %	95	90	85	80	75	70	65	60
ab BE	34	32	30	28	27	25	23	21

Notenpunkte	7	6	5	4	3	2	1	0
ab %	55	50	45	40	33	27	20	0
ab BE	20	18	16	14	12	10	7	0

Mindest-BE = aufgerundeter Anteil von 35 BE (Abitur-Skala, an Schulvorgabe anpassen). Die Probeklausur wird nicht benotet – die Notenpunkte dienen der Selbsteinschätzung.

Auswertung der Generalprobe (Abi-Training, fließt nicht in die Note ein)

- Papier-Boss 6: Selbst- und Peer-Korrektur nach den Kernpunkten in DS 38 (AB1 Korrektur und Fehlerjagd); die Lehrkraft entscheidet Streitfälle und korrigiert Stichproben. Ergebnis in BE auf dem Deckblatt – es wird nichts gebucht.
- Papier-Compiler (DS 38): aufgabeB_-Tests grün (Pflicht), aufgabeC_/aufgabeH_ als Kür; das Ergebnis des ersten Laufs wird im AB1 notiert.
- Fallen im Blick: b) mit istFrei()- und Indexgrenzen-Abfrage (Tests aufgabeB_nichtVorhanden und aufgabeB_leereStation, P1) und h) beide Fehler erkannt; Tabellen d) und f) fehlerfrei (P8).
- Team- und Kursstatistik: Strichliste P1–P10 zur Probeklausur (AB1, Teil 3) und Vergleich mit dem Warm-up-Nachtest; dazu im Lab das Fallen-Profil des Kurses (Lehrkraft-Auswertung) und „Deine Fallen“ im Profil jeder Schülerin und jedes Schülers.
- Zusätzlich im Lab: Online-Boss **„Generalprobe online“** (30 min, automatische Auswertung, bester Versuch zählt) sowie Level 6.1/6.2 (Fallenjagd I und II) – Punkte vergibt das Lab automatisch.

Musterlösungen, Kernpunkte und typische Fehler

a) Konstruktoren und Kapselung erläutern 4 BE (I 3 · II 1 · III 0)

Musterlösung

Ein **Konstruktor** ist eine besondere Operation, die beim Erzeugen eines Objekts (new) aufgerufen wird; er hat den Namen der Klasse, keinen Rückgabetyp und belegt die Attribute des neuen Objekts mit Anfangswerten.

Paketstation(standort) setzt den Standort und erzeugt eine leere dynamische Reihung faecher, eine leere Schlange auftraege und einen leeren Stapel ruecksendungen. Die Station hat danach noch keine Fächer; diese werden einzeln mit fachHinzufuegen ergänzt (z. B. Fächer mit besonderen Nummern).

Paketstation(standort, anzahlS, anzahlM, anzahlL) initialisiert dieselben Attribute und erzeugt zusätzlich anzahlS Fächer der Größe S, anzahlM der Größe M und anzahlL der Größe L, die fortlaufend nummeriert an faecher angehängt werden. Die Station Wiesenau (2, 3, 2) erhält so die Fächer 101 bis 107. Beide Konstruktoren haben denselben Namen, unterscheiden sich aber in der Parameterliste (Überladung); welcher ausgeführt wird, entscheidet der Aufruf.

„–“ bedeutet **privat**: Auf inhalt kann nur innerhalb der Klasse Fach direkt zugegriffen werden (Datenkapselung). Paketstation verwendet die **öffentliche** (+) Operation getInhalt() des Fachs und ruft für das gelieferte Paket getNummer() auf, z. B. `f.getInhalt().getNummer()` – bei einem freien Fach liefert getInhalt() null, deshalb vorher istFrei() prüfen.

Kernpunkte und BE-Verteilung (Korrekturbogen)

Kernpunkt	AFB	BE	P	erreicht
Konstruktor als besondere Operation: erzeugt ein Objekt der Klasse und belegt seine Attribute mit Anfangswerten	I	1	–	☐
Paketstation(standort): setzt den Standort, erzeugt leere dynamische Reihung, leere Schlange und leeren Stapel – Station ohne Fächer (Fächer später mit fachHinzufuegen)	I	1	P10	☐
Paketstation(standort, anzahlS, anzahlM, anzahlL): erzeugt zusätzlich die angegebene Anzahl Fächer je Größe und hängt sie an (Abb. 2: 2/3/2 → Fächer 101 bis 107); gleicher Name, verschiedene Parameterlisten (Überladung)	I	1	P10	☐
„–“: privat, nur innerhalb von Fach zugreifbar (Kapselung); Zugriff über die öffentliche (+) Operation getInhalt(), danach getNummer() von Paket; vorher istFrei() prüfen (null)	II	1	P10	☐

Alternativen / Teilpunkte: Die Überladung muss nicht benannt werden, wenn der Unterschied der Parameterlisten erläutert ist. Ohne Bezug zur Klasse Paketstation (nur allgemeine Definition): höchstens 2 BE. Für den letzten Kernpunkt genügt „über die öffentliche Operation getInhalt()“ mit Bezug zur Sendungsnummer.

Typische Fehler

- **[P10]** Konstruktoren nur genannt („es gibt zwei Konstruktoren“) statt ihre Funktion erläutert.
- **[P10]** Kein Bezug zu Abbildung 1/2 (Station Wiesenau, Fächer 101–107).
- **[P1]** Zugriff über getInhalt() ohne Hinweis, dass bei freiem Fach null geliefert wird.

b) Implementierung: Suche in einer dynamischen Reihung von Objekten 7 BE (I 1 · II 4 · III 2)

Musterlösung

Musterlösung (Java; mit javac 21 und Nachbauten der Bibliotheksklassen geprüft, Papier-Compiler-Tests aufgabeB_ 6/6 grün; Abbildung 2: P-2208 → 103, P-7310 → 101, P-5512 → 106, P-9999 → −1, leere Station → −1):

```
public int sucheFach(String sendungsnr) {
    for (int i = 0; i < faecher.getLength(); i++) {
        Fach f = (Fach) faecher.getItem(i);
        if (!f.istFrei() && f.getInhalt().getNummer().equals(sendungsnr)) {
            return f.getNummer();
        }
    }
    return -1;
}
```

Kernpunkte und BE-Verteilung (Korrekturbogen)

Kernpunkt	AFB	BE	P	erreicht
Methodenkopf passend zur Signatur (Rückgabetyp Ganzzahl, Parameter Zeichenkette)	I	1	P3	☐
Durchlauf aller Fächer mit Index 0 bis faecher.getLength() − 1	II	1	P6	☐
Zugriff mit getItem(i) und Cast (Fach)	II	1	P4	☐
istFrei() wird vor getInhalt() geprüft – kein Zugriff über null	II	1	P1	☐
Vergleich der Sendungsnummern mit equals	II	1	P5	☐
Rückgabe der Fachnummer (getNummer(), nicht der Index i) beim ersten Treffer	III	1	P3	☐
Rückgabe −1 erst nach dem vollständigen Durchlauf; Rückgabe auf jedem Pfad; leere Station ohne Laufzeitfehler	III	1	P3	☐

Alternativen / Teilpunkte: Gleichwertig: while-Schleife, for-each über faecher (`for (Object o : faecher)` mit Cast) oder Wahrheitswert gefunden. == statt equals: Kernpunkt P5 nicht erreicht (fällt im Papier-Compiler nicht auf!). Fehlender Cast: Kernpunkt P4 nicht erreicht, auch wenn der Rest richtig ist. Rückgabe des Index statt der Fachnummer: höchstens 6 BE. Pseudocode statt Java: höchstens 4 BE.

Alternative: Schleife mit Ergebnisvariable und einem return (ebenfalls 6/6 grün)

```
public int sucheFach(String sendungsnr) {
    int ergebnis = -1;
    int i = 0;
    while (ergebnis == -1 && i < faecher.getLength()) {
        Fach f = (Fach) faecher.getItem(i);
        if (!f.istFrei()) {
            Paket p = f.getInhalt();
            if (p.getNummer().equals(sendungsnr)) {
                ergebnis = f.getNummer();
            }
        }
        i++;
    }
    return ergebnis;
}
```

Typische Fehler

- **[P1]** getInhalt().getNummer() ohne istFrei()-Prüfung → NullPointerException beim ersten freien Fach (Fach 102).
- **[P3]** return −1 im else-Zweig innerhalb der Schleife – nur das erste Fach wird geprüft.
- **[P3]** Index i statt f.getNummer() zurückgegeben (2 statt 103).
- **[P6]** Schleife bis i <= getLength() → IndexOutOfBounds, wenn das Paket fehlt.
- **[P4]** Cast (Fach) vergessen – Object hat keine Methode istFrei (Compilerfehler).
- **[P5]** Vergleich der Sendungsnummern mit ==.

c) Struktogramm: Schlange auswerten und unverändert lassen 6 BE (I 1 · II 4 · III 1)

Musterlösung

Musterlösung (in Java übersetzt und mit den Papier-Compiler-Tests aufgabeC_ geprüft: 5/5 grün; M → 3, S → 1, L → 2, leere Schlange → 0, Reihenfolge unverändert):

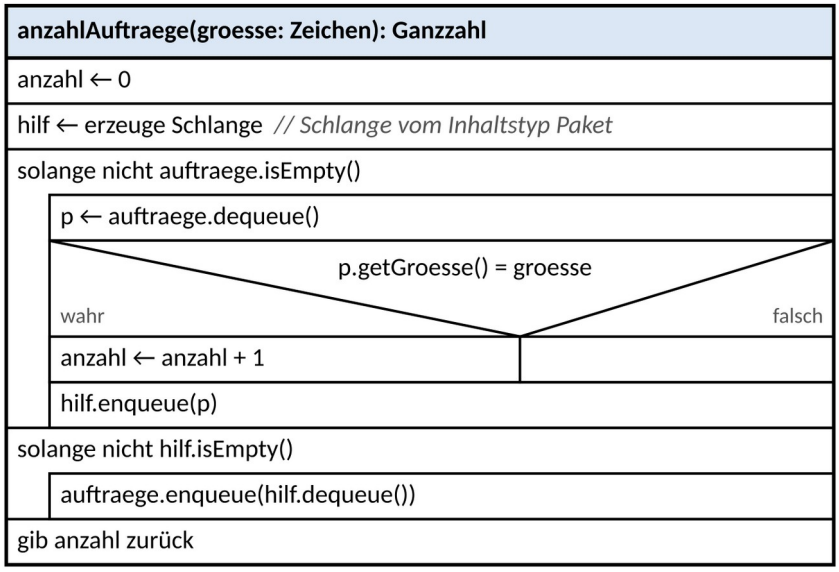


Abbildung EH.1: Struktogramm der Operation anzahlAuftraege

Kernpunkte und BE-Verteilung (Korrekturbogen)

Kernpunkt	AFB	BE	P	erreicht
Kopf mit Signatur, Zählvariable mit 0 initialisiert, Rückgabe mit „gib … zurück“	I	1	P9	☐
Hilfsschlange erzeugen (erzeuge Schlange)	II	1	–	☐
Schleife „solange nicht auftraege.isEmpty()“ mit dequeue – keine Entnahme aus leerer Schlange	II	1	P1	☐
Vergleich p.getGroesse() = groesse (Zeichen) und Zählen	II	1	P4	☐
jedes Paket in die Hilfsschlange einreihen und danach alle Pakete zurück in auftraege	II	1	P2	☐
Reihenfolge bleibt erhalten (Hilfsstruktur Schlange, kein Stapel); Notation nach EH §5 (←, Schleifen, Verzweigung, keine Java-Syntax)	III	1	P2 P9	☐

Alternativen / Teilpunkte: Gleichwertig: Zählen über eine Hilfsschlange, die erst im zweiten Durchlauf ausgewertet wird. Rotieren ohne Hilfsschlange ist nur richtig, wenn die Anzahl der Elemente vorher bestimmt wurde (dafür ist ein vollständiger Durchlauf nötig). Mit Hilfsstapel und einfachem Zurückschichten ist die Reihenfolge umgekehrt: letzter Kernpunkt nicht erreicht. Java-Code statt Struktogramm: höchstens 3 BE. Die Schreibweise „p.getGroesse() = groesse“ (Vergleich) ist nach EH §5 korrekt; „==“ ist Java-Syntax.

Typische Fehler

- **[P2]** Schlange nach dem Zählen nicht wiederhergestellt (leer).
- **[P2]** Hilfsstapel und Zurückschichten – Reihenfolge umgekehrt.
- **[P9]** Java-Syntax im Struktogramm (anzahl++, ==, while (...), return).
- **[P1]** „wiederhole ... bis auftraege.isEmpty()“ (fußgesteuert) – dequeue auf leerer Schlange.
- **[P4]** Größe mit Zeichenkette "M" statt mit dem Parameter groesse (Zeichen) verglichen.

d) Tracetabelle mit Schlange, dynamischer Reihung und Stapel 4 BE (I 2 · II 2 · III 0)

Musterlösung

Tracetabelle (erzeugt durch Ausführen einer zeilengetreuen, instrumentierten Java-Fassung von Abbildung 3; Fächer danach: 101 P-7310, 102 P-1863, 103 P-2208, 104 P-4127, 105 P-6620, 106 P-5512, 107 P-9054):

Durchlauf	p	i	gefunden	eingelegt in Fach	ruecksendungen (oben zuerst)	anzahl
vor der Schleife	–	–	–	–	–	0
1	P-4127 (M)	3	wahr	104	–	1
2	P-9054 (L)	6	wahr	107	–	2
3	P-1863 (S)	1	wahr	102	–	3
4	P-6620 (M)	4	wahr	105	–	4
5	P-3089 (M)	7	falsch	–	P-3089 (M)	4
6	P-8845 (L)	7	falsch	–	P-8845 (L), P-3089 (M)	4

Rückgabe: **4**. In Durchlauf 4 ist Fach 104 bereits belegt (Durchlauf 1), daher landet P-6620 in Fach 105. In den Durchläufen 5 und 6 läuft die innere Schleife bis i = 7 = faecher.getLength(); die Pakete kommen auf den Stapel – P-8845 liegt danach oben.

Kernpunkte und BE-Verteilung (Korrekturbogen)

Kernpunkt	AFB	BE	P	erreicht
Durchläufe 1 und 2: P-4127 (M) → i = 3, Fach 104; P-9054 (L) → i = 6, Fach 107	I	1	P8	☐
Durchläufe 3 und 4: P-1863 (S) → i = 1, Fach 102; P-6620 (M) → i = 4, Fach 105 (Fach 104 ist seit Durchlauf 1 belegt)	I	1	P8	☐
Durchläufe 5 und 6: kein freies M- bzw. L-Fach, i = 7, gefunden falsch; ruecksendungen P-3089 (M) bzw. P-8845 (L), P-3089 (M) (oben zuerst)	II	1	P8	☐
anzahl in jeder Zeile fortgeschrieben (1, 2, 3, 4, 4, 4), Rückgabe 4; alle vorgegebenen Spalten ausgefüllt	II	1	P8	☐

Alternativen / Teilpunkte: p darf ohne Größe notiert werden, wenn die Größen aus Abbildung 2 erkennbar sind. Folgefehler aus einem Durchlauf werden in den folgenden Durchläufen nicht erneut abgezogen. Stapel in falscher Reihenfolge (P-3089 oben): Kernpunkt 3 nicht erreicht.

Typische Fehler

- **[P8]** Nur die Durchläufe mit Treffer eingetragen.
- **[P8]** Fach 104 in Durchlauf 4 erneut verwendet (Belegung aus Durchlauf 1 übersehen).
- **[P8]** Stapel ruecksendungen von unten nach oben notiert.
- **[P8]** i in den Durchläufen 5 und 6 als 6 statt 7 angegeben.

e) Veränderung eines Algorithmus erläutern 2 BE (I 0 · II 1 · III 1)

Musterlösung

Die Suche nach einem freien Fach wird in eine Schleife über die Größen eingebettet: Zuerst wird wie bisher ein freies Fach der Paketgröße gesucht. Wird keines gefunden und ist die Paketgröße S, wird anschließend nach einem freien Fach der Größe M und danach L gesucht; bei M nur noch nach L. Erst wenn auch die letzte Suche erfolglos bleibt, wird das Paket auf ruecksendungen gelegt.

Weil die Größen aufsteigend durchprobiert werden, wird immer ein kleinstmögliches freies Fach verwendet. Eine einzige Suche mit der Bedingung „Fachgröße ≥ Paketgröße“ würde das erste ausreichend große Fach wählen – das ist nur dann das kleinste, wenn die Fächer nach Größe sortiert sind (beim zweiten Konstruktor der Fall, bei fachHinzufuegen nicht garantiert); sonst muss das Fach mit der kleinsten Größe über alle passenden freien Fächer bestimmt werden. Für den Vergleich der Größen (Zeichen) bietet sich eine Hilfsoperation an, die S, M, L auf 1, 2, 3 abbildet (wie getVolumen).

Kernpunkte und BE-Verteilung (Korrekturbogen)

Kernpunkt	AFB	BE	P	erreicht
Ohne Treffer bei gleicher Größe: weitere Suche nach einem freien Fach der nächstgrößeren Größe (S → M → L) bzw. Bedingung „Fachgröße ≥ Paketgröße“; Rücksendung erst, wenn auch diese Suche erfolglos ist	II	1	P10	☐
Kleinstes passendes Fach: Suche größenweise aufsteigend in getrennten Durchläufen (nicht „erstes Fach mit ausreichender Größe“, da die Fächer nicht nach Größe sortiert sein müssen) oder Minimum der Größe über alle passenden freien Fächer bestimmen; Größenvergleich z. B. über eine Hilfsoperation Größe → Volumen	III	1	P10	☐

Alternativen / Teilpunkte: Auch eine Erläuterung über eine einzige Suche mit „Fachgröße ≥ Paketgröße“ ist richtig, wenn begründet wird, warum dabei das kleinste Fach gewählt wird (z. B. Sortierung der Fächer nach Größe oder Merken des bisher kleinsten passenden Fachs). Code oder Struktogramm sind nicht verlangt, aber zulässig.

Typische Fehler

- **[P10]** Nur „man muss auch größere Fächer nehmen“ – ohne Ablauf und ohne Bezug zum kleinsten Fach.
- **[P10]** Vorschlag, die Größe einfach zu ignorieren (Paket S landet im ersten freien Fach, auch L).

f) Rekursionstabelle und Prinzip der Rekursion 5 BE (I 2 · II 3 · III 0)

Musterlösung

Rekursionstabelle (erzeugt durch eine instrumentierte Java-Fassung von Abbildung 5):

Rekursions-tiefe *)	Stapel s beim Aufruf (oben zuerst)	platz	p	p.getVolumen()	Rückgabe
0 (Erstaufruf)	P-2741 (M), P-0412 (S), P-7781 (L), P-5236 (S), P-1190 (M)	8	P-2741 (M)	2	4
1	P-0412 (S), P-7781 (L), P-5236 (S), P-1190 (M)	6	P-0412 (S)	1	3
2	P-7781 (L), P-5236 (S), P-1190 (M)	5	P-7781 (L)	3	2
3	P-5236 (S), P-1190 (M)	2	P-5236 (S)	1	1
4	P-1190 (M)	1	P-1190 (M)	2	0

Rückgabe des Erstaufrufs: **4**; danach enthält s nur noch P-1190 (M).

Prinzip der Rekursion: sammle löst das Problem, indem sie sich selbst für ein kleineres Teilproblem aufruft: Passt das oberste Paket, wird es vom Stapel genommen, und für den kürzeren Stapel mit um sein Volumen verringertem platz wird erneut sammle aufgerufen (Rekursionsschritt). Die Rekursion endet in einem der beiden **Basisfälle** – der Stapel ist leer oder das oberste Paket passt nicht mehr –, die ohne weiteren Aufruf 0 liefern. Da jeder Aufruf den Stapel verkürzt, wird ein Basisfall sicher erreicht. Das Ergebnis entsteht auf dem Rückweg: Jede Ebene liefert 1 + Ergebnis des rekursiven Aufrufs.

Kernpunkte und BE-Verteilung (Korrekturbogen)

Kernpunkt	AFB	BE	P	erreicht
Tiefen 0 bis 2 korrekt (Erstaufruf Tiefe 0; s, platz 8, 6, 5; p und Volumen)	I	1	P7 P8	☐
Tiefen 3 und 4 korrekt; Abbruch in Tiefe 4, weil P-1190 (M) nicht passt (2 > 1)	I	1	P7 P8	☐
Rückgaben von innen nach außen 0, 1, 2, 3, 4; Rückgabe des Erstaufrufs 4	II	1	P7	☐
Erläuterung: sammle ruft sich selbst mit einem kleineren Problem auf (Stapel um ein Paket kürzer, platz um dessen Volumen kleiner)	II	1	P10	☐
Basisfälle (leerer Stapel; oberstes Paket passt nicht) beenden die Rekursion mit 0; das Ergebnis entsteht auf dem Rückweg als 1 + Ergebnis des rekursiven Aufrufs	II	1	P7	☐

Alternativen / Teilpunkte: Die Spalte „Stapel s“ darf mit Abkürzungen (z. B. M, S, L, S, M) geführt werden. Rekursionstiefe ab 1 gezählt: Kernpunkt 1 nicht erreicht (P7), Folgefehler nicht erneut abziehen. Für die Erläuterung genügen Basisfall, Selbstaufruf mit verkleinertem Problem und Zusammensetzen des Ergebnisses mit Bezug zu sammle. Ergänzend (nicht verlangt): sammle(s, 10) leert den Stapel vollständig und endet in Tiefe 5 im Basisfall „Stapel leer“ (Rückgabe 5).

Typische Fehler

- **[P7]** Rekursionstiefe beim Erstaufruf mit 1 begonnen.
- **[P7]** Rückgaben von oben nach unten (4 in Tiefe 4) eingetragen.
- **[P8]** Stapelinhalt nach statt vor dem pop eingetragen bzw. platz falsch fortgeschrieben.
- **[P10]** Prinzip nur allgemein („Methode ruft sich selbst auf“) ohne Basisfälle und Bezug zu sammle.

g) Rekursionstiefe begründen 2 BE (I 0 · II 1 · III 1)

Musterlösung

sammle ruft sich nur dann selbst auf, wenn das oberste Paket passt, also p.getVolumen() ≤ platz gilt. Der neue Wert platz − p.getVolumen() ist deshalb nie negativ. Da jedes Paket mindestens das Volumen 1 hat (S), ist platz in jeder tieferen Ebene um mindestens 1 kleiner als in der darüberliegenden.

Für einen Aufruf in Rekursionstiefe t (Erstaufruf t = 0) gilt daher 0 ≤ platz_t ≤ platz − t, also t ≤ platz. Spätestens in Tiefe platz ist kein Platz mehr frei, und der Basisfall „Paket passt nicht“ (oder „Stapel leer“) beendet die Rekursion. Die Schranke wird nur erreicht, wenn alle eingeladenen Pakete die Größe S haben; wie viele Pakete im Stapel liegen, spielt keine Rolle (bei wenigen Paketen endet die Rekursion durch den leeren Stapel noch früher).

Kernpunkte und BE-Verteilung (Korrekturbogen)

Kernpunkt	AFB	BE	P	erreicht
Selbstaufruf nur, wenn das oberste Paket passt (getVolumen() ≤ platz); jedes Paket hat ein Volumen von mindestens 1 – platz sinkt mit jeder Rekursionsebene um mindestens 1	II	1	P10	☐
platz wird nie negativ; in Tiefe t ist platz höchstens um t kleiner als beim Erstaufruf → t ≤ platz (Erstaufruf Tiefe 0); Gleichheit nur bei lauter S-Paketen; die Anzahl der Pakete spielt für die Schranke keine Rolle	III	1	P7 P10	☐

Alternativen / Teilpunkte: Eine Begründung am Beispiel (Tiefe 4 bei platz = 8) allein reicht nicht (P10: „begründen“ verlangt die allgemeine Argumentation). Der Hinweis „höchstens so tief, wie Pakete im Stapel liegen“ ist richtig, beantwortet aber die Frage nicht.

Typische Fehler

- **[P10]** Nur am Beispiel gezeigt (Tiefe 4 ≤ 8) statt allgemein begründet.
- **[P7]** Tiefe ab 1 gezählt und deshalb „höchstens platz + 1“ gefolgert.

h) Fehleranalyse einer rekursiven Methode 5 BE (I 1 · II 3 · III 1)

Musterlösung

Analyse (geprüft durch Ausführen von Abbildung 8): Erstaufruf i = 0, Fach 101 ist belegt → anzahl = 0. Der rekursive Aufruf in Zeile 10 läuft zwar bis i = 6 (insgesamt 7 Aufrufe), sein Ergebnis wird aber verworfen – zurückgegeben wird nur der Wert für Fach 101: 0 (für M, L und S).

Fehler 1 (Zeile 10): Das Ergebnis des rekursiven Aufrufs wird nicht verwendet; es zählt nur das Fach an Index i.

Fehler 2 (Zeile 2): Der Basisfall greift schon bei i = getLength() − 1 und liefert 0, ohne das letzte Fach zu prüfen. Fach 107 (frei, L) wird nie gezählt – auch nach Korrektur von Fehler 1 bliebe freieFaecher('L', 0) bei 0.

Station ohne Fächer: getLength() − 1 = −1; für i = 0 greift der Basisfall nicht, getItem(0) führt zu einem Laufzeitfehler (IndexOutOfBoundsException). Mit dem korrigierten Basisfall i >= getLength() wird sofort 0 geliefert.

Korrektur (Papier-Compiler-Tests aufgabeH_ 6/6 grün; M → 2, L → 1, S → 1, leere Station → 0):

```
public int freieFaecher(char groesse, int i) {
    if (i >= faecher.getLength()) {
        return 0;
    }
    Fach f = (Fach) faecher.getItem(i);
    int anzahl = freieFaecher(groesse, i + 1);
    if (f.istFrei() && f.getGroesse() == groesse) {
        anzahl = anzahl + 1;
    }
    return anzahl;
}
```

Kernpunkte und BE-Verteilung (Korrekturbogen)

Kernpunkt	AFB	BE	P	erreicht
Fehler 1 (Zeile 10): Ergebnis des rekursiven Aufrufs wird nicht verwendet – es zählt nur das Fach an Index i (Erstaufruf: Fach 101 ist belegt → 0)	I	1	P7	☐
Fehler 2 (Zeile 2): Basisfall schon bei i = getLength() − 1 – das letzte Fach (107, frei, Größe L) wird nie geprüft	II	1	P6 P7	☐
Korrektur: Basisfall i == faecher.getLength() bzw. i >= faecher.getLength() liefert 0; Ergebnis des rekursiven Aufrufs addieren	II	1	P7	☐
korrigierte Methode vollständig und lauffähig (Cast, Bedingung, Rückgabe auf jedem Pfad, rekursiv – keine Schleife)	II	1	P3 P4	☐
Station ohne Fächer: getLength() − 1 = −1, der Basisfall greift für i = 0 nicht → getItem(0) → Laufzeitfehler (IndexOutOfBoundsException); die korrigierte Methode liefert 0	III	1	P1	☐

Alternativen / Teilpunkte: Fehler dürfen in anderer Reihenfolge genannt werden. Die Korrektur „Basisfall i == getLength()“ ist gleichwertig zu „i >= getLength()“. Eine Korrektur mit Schleife statt Rekursion verfehlt die Vorgabe (Kernpunkt 4 nicht erreicht). Nur einen der beiden Fehler korrigiert: höchstens 3 BE.

Gleichwertige minimale Korrektur (Zeilen 2 und 10 geändert, ebenfalls 6/6 grün)

```
public int freieFaecher(char groesse, int i) {
    if (i == faecher.getLength()) {
        return 0;
    }
    Fach f = (Fach) faecher.getItem(i);
    int anzahl = 0;
    if (f.istFrei() && f.getGroesse() == groesse) {
        anzahl = 1;
    }
    return anzahl + freieFaecher(groesse, i + 1);
}
```

Typische Fehler

- **[P7]** Nur „Basisfall falsch“ erkannt, das verworfene Ergebnis übersehen.
- **[P6]** Basisfall auf i > getLength() korrigiert → getItem(getLength()) → Laufzeitfehler.
- **[P7]** Rekursiver Aufruf mit i statt i + 1 → Endlosrekursion (StackOverflowError).
- **[P1]** Station ohne Fächer nicht untersucht oder „liefert 0“ behauptet.

Anhang: Papier-Compiler und Testklasse

Die Datei **AQ_W6_Probeklausur_Papier-Compiler.txt** (Gerüst) wird erst in DS 38 ausgegeben: Workspace „W6 Papier-Compiler“ anlegen, Datei einfügen, Bibliothek „Niedersachsen“ aktivieren (Rechtsklick auf den Workspace → Einstellungen → Häkchen) und über „An Klasse austeilen“ verteilen. Die Lernenden tippen b) (Pflicht) sowie c) und h) (Kür; c) als Übersetzung des eigenen Struktogramms) **unverändert** in den Abschnitt „DEIN TEIL“ ab und starten die Testklasse ProbeklausurTest im Reiter „Testrunner“. Erst danach wird korrigiert. Die Musterlösung steht in **AQ_W6_Probeklausur_Papier-Compiler_LSG.txt** (17/17 grün; im Gerüst 0/17).

Vor Einsatz prüfen

Bibliothek „Niedersachsen“ aktivieren; Casts (Fach), (Paket) und char-Vergleiche im Dialekt der Online-IDE, den JUnit-Dialekt (parameterloser Konstruktor, @Test, globale assertEquals mit Meldung als letztem Argument) und das Verhalten des Testrunners bei NullPointerException, IndexOutOfBoundsException und StackOverflowError (Mutanten unten) mit einem Probedurchlauf testen. Die Hilfsmethode wiesenau() der Testklasse ist private und ohne @Test.

Testklasse ProbeklausurTest (17 Tests: aufgabeB_ 6, aufgabeC_ 5, aufgabeH_ 6)

```
class ProbeklausurTest {

    private Fach[] f;

    public ProbeklausurTest() {
    }

    // Station Wiesenau aus Abbildung 2: Faecher 101 bis 107,
    // belegt sind 101, 103 und 106; sechs Zustellauftraege
    private Paketstation wiesenau() {
        f = new Fach[7];
        f[0] = new Fach(101, 'S');
        f[1] = new Fach(102, 'S');
        f[2] = new Fach(103, 'M');
        f[3] = new Fach(104, 'M');
        f[4] = new Fach(105, 'M');
        f[5] = new Fach(106, 'L');
        f[6] = new Fach(107, 'L');
        Paketstation st = new Paketstation("Wiesenau");
        for (int i = 0; i < 7; i++) {
            st.fachHinzufuegen(f[i]);
        }
        f[0].einlegen(new Paket("P-7310", "Yilmaz", 'S'));
        f[2].einlegen(new Paket("P-2208", "Brandt", 'M'));
        f[5].einlegen(new Paket("P-5512", "Okafor", 'L'));
        st.auftragAnnehmen(new Paket("P-4127", "Kaya", 'M'));
        st.auftragAnnehmen(new Paket("P-9054", "Nowak", 'L'));
        st.auftragAnnehmen(new Paket("P-1863", "Schulz", 'S'));
        st.auftragAnnehmen(new Paket("P-6620", "Ahrens", 'M'));
        st.auftragAnnehmen(new Paket("P-3089", "Lange", 'M'));
        st.auftragAnnehmen(new Paket("P-8845", "Petrov", 'L'));
        return st;
    }

    // ------------------------ Aufgabe b) ----------------------------

    @Test
    public void aufgabeB_beispiel() {
        Paketstation st = wiesenau();
```

```
        assertEquals(103, st.sucheFach("P-2208"), "P-2208 liegt in Fach 103 (Fachnummer, nicht
Index!)");
    }

    @Test
    public void aufgabeB_erstesFach() {
        Paketstation st = wiesenau();
        assertEquals(101, st.sucheFach("P-7310"), "P-7310 liegt im ersten Fach 101 (Index 0)");
    }

    @Test
    public void aufgabeB_hinterFreienFaechern() {
        Paketstation st = wiesenau();
        assertEquals(106, st.sucheFach("P-5512"), "P-5512 liegt in Fach 106 - davor liegen freie
Faecher (istFrei?)");
    }

    @Test
    public void aufgabeB_nichtVorhanden() {
        Paketstation st = wiesenau();
        assertEquals(-1, st.sucheFach("P-9999"), "P-9999 liegt in keinem Fach: -1 (alle Faecher
pruefen, Index bis getLength()-1)");
    }

    @Test
    public void aufgabeB_leereStation() {
        Paketstation st = new Paketstation("Leer");
        assertEquals(-1, st.sucheFach("P-2208"), "Station ohne Faecher: -1 ohne Laufzeitfehler");
    }

    @Test
    public void aufgabeB_letztesFach() {
        Paketstation st = wiesenau();
        st.zustellen();
        assertEquals(107, st.sucheFach("P-9054"), "nach zustellen() liegt P-9054 im letzten Fach
107");
        assertEquals(-1, st.sucheFach("P-8845"), "P-8845 ist eine Ruecksendung und liegt in
keinem Fach");
    }

    // ------------------------ Aufgabe c) ----------------------------

    @Test
    public void aufgabeC_beispielM() {
        Paketstation st = wiesenau();
        assertEquals(3, st.anzahlAuftraege('M'), "drei Auftraege der Groesse M");
    }

    @Test
    public void aufgabeC_groessenSundL() {
        Paketstation st = wiesenau();
        assertEquals(1, st.anzahlAuftraege('S'), "ein Auftrag der Groesse S");
        assertEquals(2, st.anzahlAuftraege('L'), "zwei Auftraege der Groesse L");
    }

    @Test
    public void aufgabeC_leereSchlange() {
        Paketstation st = new Paketstation("Leer");
        assertEquals(0, st.anzahlAuftraege('M'), "keine Auftraege: 0 ohne Laufzeitfehler
(isEmpty?)");
```

```
    }

    @Test
    public void aufgabeC_unveraendert() {
        Paketstation st = wiesenau();
        int n = st.anzahlAuftraege('M');
        assertEquals(3, n, "drei Auftraege der Groesse M");
        assertEquals("P-4127 P-9054 P-1863 P-6620 P-3089 P-8845", st.auftraegeAlsText(),
            "Schlange auftraege danach unveraendert (gleiche Pakete, gleiche Reihenfolge)");
    }

    @Test
    public void aufgabeC_zweimalAufrufen() {
        Paketstation st = wiesenau();
        st.anzahlAuftraege('L');
        assertEquals(2, st.anzahlAuftraege('L'), "zweiter Aufruf liefert dasselbe Ergebnis");
    }

    // ------------------------ Aufgabe h) ----------------------------

    @Test
    public void aufgabeH_beispielM() {
        Paketstation st = wiesenau();
        assertEquals(2, st.freieFaecher('M', 0), "freie M-Faecher: 104 und 105");
    }

    @Test
    public void aufgabeH_letztesFachZaehlt() {
        Paketstation st = wiesenau();
        assertEquals(1, st.freieFaecher('L', 0), "freies L-Fach 107 ist das letzte Fach
(Basisfall!)");
    }

    @Test
    public void aufgabeH_groesseS() {
        Paketstation st = wiesenau();
        assertEquals(1, st.freieFaecher('S', 0), "freies S-Fach: 102");
    }

    @Test
    public void aufgabeH_abIndex() {
        Paketstation st = wiesenau();
        assertEquals(1, st.freieFaecher('M', 4), "ab Index 4 ist nur noch Fach 105 ein freies M-
Fach");
        assertEquals(0, st.freieFaecher('L', 7), "ab Index 7 gibt es keine Faecher mehr");
    }

    @Test
    public void aufgabeH_leereStation() {
        Paketstation st = new Paketstation("Leer");
        assertEquals(0, st.freieFaecher('M', 0), "Station ohne Faecher: 0 ohne Laufzeitfehler");
    }

    @Test
    public void aufgabeH_nachZustellen() {
        Paketstation st = wiesenau();
        st.zustellen();
        assertEquals(0, st.freieFaecher('M', 0), "nach zustellen() ist kein M-Fach mehr frei");
        assertEquals(0, st.freieFaecher('L', 0), "nach zustellen() ist kein L-Fach mehr frei");
    }
```

```
}
```

Kontrolle der Testklasse mit typischen Fehlvarianten (Mutationstests, javac 21 + Nachbauten der Bibliotheksklassen)

Teil	P	Fehlvariante	Tests grün	Ergebnis
b)	P1	keine Abfrage istFrei() (getInhalt() auf freiem Fach)	3/6	erkannt
b)	P1	Bedingungen vertauscht (getInhalt() vor istFrei())	3/6	erkannt
b)	P6	Schleife bis i <= getLength()	3/6	erkannt
b)	P6	Startindex 1	5/6	erkannt
b)	P3	Index statt Fachnummer zurückgegeben	2/6	erkannt
b)	P3	return -1 im else-Zweig in der Schleife	3/6	erkannt
b)	P3	return -1 nach der Schleife fehlt	Compilerfehler	erkannt
b)	P4	Cast (Fach) fehlt	Compilerfehler	erkannt
b)	P5	== statt equals	6/6	nicht erkennbar
c)	P2	Schlange nicht wiederhergestellt	2/5	erkannt
c)	P2	Hilfsstapel statt Hilfsschlange (Reihenfolge umgedreht)	4/5	erkannt
c)	P2	Zurückschichten in derselben Schleife (Endlosschleife)	1/5	erkannt
c)	P1	fußgesteuert: dequeue ohne Leerabfrage	4/5	erkannt
c)	P4	Vergleich mit Zeichenkette "M" statt Zeichen	Compilerfehler	erkannt
c)	P4	Cast (Paket) fehlt	Compilerfehler	erkannt
h)	P7	unveränderte Methode aus Abbildung 8 (beide Fehler)	1/6	erkannt
h)	P7	nur Basisfall korrigiert, Ergebnis weiter nicht verwendet	3/6	erkannt
h)	P6	nur Ergebnis verwendet, Basisfall i == getLength() − 1 bleibt	2/6	erkannt
h)	P7	Basisfall fehlt	0/6	erkannt
h)	P7	rekursiver Aufruf mit i statt i + 1	1/6	erkannt
h)	P6	Basisfall i > getLength()	0/6	erkannt
h)	P7	Ergebnis nicht verwendet (Aufruf ohne Zuweisung)	3/6	erkannt
h)	P3	return fehlt im Basisfall-Pfad (nur if ohne return)	Compilerfehler	erkannt

„Tests grün“ bezieht sich auf die Tests der jeweiligen Teilaufgabe; „Compilerfehler“ bedeutet: Die Online-IDE meldet den Fehler schon vor dem Testlauf. **Nicht erkennbar:** P5 (== statt equals) – die Online-IDE vergleicht Zeichenketten mit == inhaltlich, und in echtem Java sind gleiche Literale identisch. Ebenfalls nicht prüfbar: ob h) wirklich rekursiv ist und ob c) der Struktogramm-Notation folgt. Diese Punkte bleiben Aufgabe des Korrekturbogens.